IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.Doi Number

TwinShield-Fusion: Digital-Twin Residual Gating for Detection of Unseen Attacks on Automotive CAN Buses

FIRST A. AUTHOR[1], SECOND B. AUTHOR[2], and THIRD C. AUTHOR[3]

[1]Department of Computer Science and Engineering, [Institution], [City], [Country]
[2]Department of Computer Science and Engineering, [Institution], [City], [Country]
[3]Department of Computer Science and Engineering, [Institution], [City], [Country]

Corresponding author: First A. Author (e-mail: author@institution.edu).

This work was carried out as part of an undergraduate research project. The GEM-CAN dataset is distributed under CC-BY-NC-3.0 and was obtained from Zenodo (10.5281/zenodo.17834776).

ABSTRACT An electric vehicle (EV) distributes braking, steering and propulsion commands across dozens of electronic control units (ECUs) communicating over the Controller Area Network (CAN), a broadcast bus whose frames carry no sender identity and no authentication. Any node reaching the bus can inject commands, and because the protocol cannot be revised without replacing every ECU, intrusion detection is the practical defence. Published detectors are scored on a fixed attack catalogue, so an attack outside it is assigned to whichever known class fits least badly. We propose TwinShield-Fusion, pairing a supervised classifier with a gated recurrent unit (GRU) digital twin fitted on normal traffic alone. Its payload prediction residual is standardised per identifier and per byte, then thresholded at a calibrated false-positive rate. On GEM-CAN, 143,444 frames from a GEM e6 EV, with one attack class quarantined from every training stage, the baselines detect 0.84%, 0.00% and 25.21% of it; TwinShield-Fusion detects 100.00% at 2.26% false positives.

INDEX TERMS Anomaly detection, automotive security, controller area network, digital twin, intrusion detection systems, open-set recognition, vehicular networks, zero-day attacks.

I. INTRODUCTION

The Controller Area Network (CAN), standardised as ISO 11898-1 [1], has been the backbone of in-vehicle communication for three decades, and it was designed for a threat model that no longer holds. A CAN frame carries an arbitration identifier, a data length code and up to eight payload bytes; it carries no sender identity, no message authentication code and no encryption. Arbitration is decided by the numeric value of the identifier alone, so an electronic control unit (ECU) that transmits a sufficiently low identifier wins the bus every time it contends for it. These properties are intrinsic to the protocol rather than defects in any implementation.

The electric vehicle (EV) raises the stakes. Braking, steering and propulsion in an EV are issued as digital commands over the same bus that carries diagnostics and telemetry, and drive-by-wire platforms expose those commands to any controller attached to the network. Koscher et al. [2] showed that physical access to the bus permits arbitrary control of safety functions, Checkoway et al. [3] extended the result to remote entry points, and Miller and Valasek [4] drove a Jeep Cherokee off a highway from a laptop. Because the protocol cannot be changed without replacing every ECU on the vehicle, the practical response has been to monitor the bus and raise an alarm when traffic departs from expected behaviour. This is the role of an in-vehicle intrusion detection system (IDS).

A large body of work applies machine learning (ML) and deep learning (DL) to that monitoring problem, and the reported numbers are remarkable: accuracies above 99% are routine [5]–[8]. Those numbers are also, in a specific sense, uninformative. They are closed-set results. The model is trained on a catalogue of attacks, tested on the same catalogue, and scored on how often it names the right member of it. That

protocol leaves open what happens when an attack arrives that was absent from the catalogue, which is the situation a deployed detector will eventually face.

The gap is structural. A supervised classifier is bounded by its label set. With a softmax over k trained classes it can only emit one of those k labels, so an attack outside the set is assigned to whichever of them fits least badly. This is the open-set recognition problem [9], and the complementary route to it is anomaly detection against a model of normal behaviour [10]. For traffic that does not resemble a flood, that is almost always the normal class. Additional convolutional or recurrent capacity leaves this bound in place, because the missing ingredient is a reference for what the bus ought to be doing, independent of any catalogued attack.

We build that reference as a digital twin. A gated recurrent unit (GRU) network is fitted exclusively on normal traffic and learns to predict the next payload on each arbitration identifier from the recent history of that identifier. The residual between prediction and observation measures how far the bus has drifted from learnt-normal behaviour, and it is defined for any frame whatsoever, including one belonging to an attack that has never been labelled. A threshold on that residual, calibrated on held-out normal frames alone, forms a second decision channel alongside the supervised classifier.

The contributions of this work are as follows.

- A leave-one-attack-out evaluation protocol for CAN intrusion detection that quarantines an attack class at four stages: the label space, every training context window, the digital twin's per-ECU history, and the calibration set of the anomaly threshold. We show that removing the class from the labels alone leaves residual contamination, and we quantify how much.
- A two-view residual anomaly gate that standardises the digital-twin residual per arbitration identifier and per payload byte, combining a frame-level view with an ECU-stream view. We show that this standardisation, rather than a global threshold, is what makes a single bus-wide operating point usable.
- An empirical demonstration that the closed-set benchmark on GEM-CAN is saturated: the four architectures compared in this work lie within 0.0031 macro-F1 of one another, while the same four separate by more than 74 percentage points on a quarantined attack class.
- A measured boundary for the method. Detection requires the attack to raise the twin's residual above learnt-normal; held out in turn, steering tampering stays inside the range the twin already accepts for its identifier. We report this limit with the same prominence as the positive result and quantify the condition under which the gate applies.

The remainder of this paper is organised as follows. Section II reviews machine learning, deep learning, unsupervised and specification-based intrusion detection for CAN, together with the public datasets on which they are evaluated, and summarises the limitations of that body of work. Section III describes the GEM-CAN dataset and the threat model. Section IV presents the proposed TwinShield-Fusion architecture, following the order of its processing stages. Section V specifies the experimental setup, including the leakage-controlled split and the leave-one-attack-out protocol. Section VI reports closed-set results, quarantined-class detection, false-positive cost and runtime. Section VII discusses the limitations of the method, and Section VIII concludes.

II. RELATED WORK

Research on in-vehicle intrusion detection divides along two axes. The first is the learning paradigm: supervised methods fit a decision boundary against labelled attack traffic, whereas unsupervised and specification-based methods model normal behaviour and flag departures from it. The second is the feature level: frame-level methods read arbitration identifiers, payload bytes and inter-arrival times directly, while signal-level methods first decode payloads into physical signals. Rajapaksha et al. [11] survey the field and report that the great majority of published systems are supervised, frame-level, and evaluated on a small number of public datasets under closed-set assumptions. We organise this section along the same axes, name the dataset each study used, and close with a summary of what the body of work leaves unresolved.

A. CLASSICAL MACHINE LEARNING METHODS

Early learning-based detectors applied conventional classifiers to hand-engineered frame features. Bari et al. [12] compared a support vector machine, a decision tree and a k-nearest-neighbour classifier on the HCRL Car-Hacking dataset, which contains denial-of-service (DoS), fuzzing and spoofing captures from a Kia Soul and a Chevrolet Spark, and reported accuracies above 99% for the tree-based models. Lee et al. [13] took a protocol-level route with OTIDS, measuring the offset ratio and time interval between a remote frame and the response it elicits; the associated OTIDS dataset, captured from a KIA Soul, has since become a common benchmark. Müter and Asaj [14] dispensed with learning altogether and monitored the entropy of the message stream, an approach that detects volumetric injection on any vehicle without training data.

These methods are inexpensive and interpretable, and they transfer poorly to attacks that preserve the statistical profile of the bus. Each is evaluated on captures whose attack classes are known in advance, so the reported accuracies describe recognition of catalogued attacks rather than detection of novel ones.

B. DEEP LEARNING METHODS

Deep models removed the feature-engineering step. Kang and Kang [5] applied a deep neural network to CAN payloads on a simulated OCTANE capture and reported detection rates above 97%. Song et al. [6] arranged 29 consecutive frames into a two-dimensional grid and applied a reduced Inception-

ResNet to the Car-Hacking dataset, exploiting the temporal structure of injection. Seo et al. [7] proposed GIDS, in which a generative adversarial network synthesises additional attack samples for training, again on Car-Hacking. Lokman et al. [8] review this family and note its uniform dependence on labelled attack traffic.

Signal-level deep models form a more recent strand. Shahriar et al. [15] proposed CANShield, a framework of convolutional autoencoders operating on decoded signal time-series at several time scales, evaluated on the simulated SynCAN dataset and on the real-vehicle ROAD dataset, and targeting masquerade attacks that frame-level methods miss. CANShield trains on attack-free traffic, which places it, like our own gate, outside the supervised family; its evaluation nonetheless reports detection per known attack type rather than under an explicit quarantine.

C. UNSUPERVISED AND PREDICTION-BASED METHODS

A third line models normal traffic and thresholds the departure from it. Taylor et al. [16] train a long short-term memory (LSTM) network to predict the next payload on each identifier and threshold the prediction error, using their own capture from a production sedan; this is the closest antecedent of the mechanism we adopt. Hanselmann et al. [17] extend the idea to CANet, an LSTM-and-autoencoder architecture over multiple decoded signals trained on attack-free traffic and evaluated on SynCAN and on a real capture. Cho and Shin [18] take an orthogonal route and fingerprint ECUs by clock skew, identifying the transmitter rather than the content, which detects a masquerading ECU but not a compromised genuine one.

Our contribution relative to [16] and [17] is the evaluation rather than the mechanism. Both works report detection on attacks absent from training, and both report it for the unsupervised channel in isolation. Neither places that channel beside supervised baselines under an identical, auditable quarantine, which is what isolates how much of the detection is attributable to independence from the label.

D. DIGITAL TWINS FOR SECURITY MONITORING

A digital twin is a model kept in correspondence with a physical system [19]. Twins have been proposed for intrusion detection in industrial control settings, where process physics constrains which sensor readings are plausible. CAN payloads on a road vehicle are less constrained than a chemical process, yet the principle survives: an ECU that repeats a predictable message pattern can be modelled, and departures from that model constitute evidence. What distinguishes a twin from a generic autoencoder in this setting is that it is conditioned on the identity of the transmitting ECU, so each stream is judged against its own learnt behaviour.

E. PUBLIC DATASETS FOR CAN INTRUSION DETECTION

Progress in this field is bounded by the available captures. Car-Hacking [6] supplies DoS, fuzzing and two spoofing classes from two passenger vehicles and remains the most widely used benchmark; OTIDS [13] adds remote-frame and fuzzing attacks on a third. SynCAN [17] is simulated, which permits signal-level ground truth at the cost of realism. Verma et al. [20] introduced ROAD, 3.5 hours of real traffic covering 33 attacks including masquerade scenarios verified on the vehicle, and documented how the convenience of earlier datasets has shaped and limited the field. GEM-CAN [21], [22], used here, is the first public capture from an autonomous electric vehicle and is described in Section III.

F. SUMMARY AND OPEN PROBLEMS

Table I summarises the studies discussed above by method, by the dataset on which each was evaluated, and by the assumption that bounds what its reported figures can establish. Three observations follow from it.

TABLE I
SUMMARY OF REPRESENTATIVE CAN INTRUSION DETECTION STUDIES

Study	Method	Dataset	Evaluation limitation
Kang and Kang [5]	Deep neural net	OCTANE (sim.)	Closed set
Song et al. [6]	Inception-ResNet	Car-Hacking	Closed set
Seo et al. [7]	GAN-augmented CNN	Car-Hacking	Closed set
Bari et al. [12]	SVM, tree, kNN	Car-Hacking	Closed set
Lee et al. [13]	Remote-frame offset	OTIDS	Known types only
Müter and Asaj [14]	Message entropy	Own capture	Volumetric only
Shahriar et al. [15]	Signal autoencoders	SynCAN, ROAD	No quarantine
Taylor et al. [16]	LSTM prediction error	Own capture	No baseline comparison
Hanselmann et al. [17]	CANet autoencoder	SynCAN, real	No baseline comparison
Cho and Shin [18]	Clock-skew fingerprint	Own capture	Masquerading ECU only
This work	Head + twin residual gate	GEM-CAN	Single vehicle capture

First, reported accuracy is uniformly high and uniformly closed-set. Every supervised entry in Table I trains and tests on the same attack catalogue, so the figures measure recognition of catalogued attacks and are silent about attacks outside them. Second, evaluation is concentrated on two or three public captures, most of them dominated by volumetric injection, where the arbitration identifier alone is close to sufficient; we quantify that shortcut on GEM-CAN in Section III-C. Third, the unsupervised entries train on attack-free traffic but report their results separately from supervised baselines, which leaves the comparison that matters for deployment unmade.

These three observations define the gap this paper addresses. We place a supervised classifier and an attack-agnostic residual gate on the same data, under one quarantine

applied at four stages, and report both under a metric that is defined irrespective of label space.

III. DATASET AND THREAT MODEL

A. DATASET OVERVIEW

We evaluate on GEM-CAN [21], [22], a public capture recorded by Tavasoli et al. from a Polaris GEM e6 autonomous electric vehicle on a closed rural test track. The vehicle runs the Autoware autonomous driving stack over a PACMod drive-by-wire interface on a 250 kbit/s CAN bus, and traffic was recorded with a PCAN-USB interface. Two captures are supplied: one of nominal autonomous driving, and one during which attacks were physically injected onto the live bus while the vehicle was stationary in autonomous mode, for safety. Every frame carries a timestamp, an arbitration identifier, a data length code, eight payload bytes and an attack-type label.

Three properties make this capture suitable for the question we ask. It is real rather than simulated, so payload dynamics reflect genuine ECU behaviour. It comes from an electric, drive-by-wire platform, so the brake and steering commands under attack are the commands that actually actuate the vehicle. And it contains both a volumetric attack and two low-rate tampering attacks on legitimate identifiers, which lets a single dataset exercise both the easy and the difficult detection regimes. Table II gives the composition. We make no claim about attack classes beyond the three the dataset contains.

TABLE II
COMPOSITION OF THE GEM-CAN CAPTURE

Class	Frames	Arbitration IDs used
Normal	101,039	13 distinct IDs
DoS	41,080	00000000 only
Brake tampering	663	18FF00F9 (shared with normal)
Steering tampering	662	18FF70F9 (shared with normal)
Total	143,444	—

The two attack families differ in a way that governs everything that follows. The denial-of-service (DoS) attack floods the bus with arbitration identifier 00000000. Because CAN arbitration is won by the numerically lowest identifier, this identifier takes priority over every legitimate message on the vehicle, and genuine traffic is starved; the source capture records bus load rising from 20% to 100% during the flood. It is also trivially separable: identifier 00000000 is transmitted by no legitimate ECU on this vehicle, so the identifier alone identifies the attack.

Brake and steering tampering are low-rate. They inject a small number of forged frames on identifiers 18FF00F9 and 18FF70F9, both of which carry genuine traffic throughout the capture, and the recorded effect includes a steering-lock delay of roughly 20 seconds. The identifier is therefore uninformative as a discriminator, and detection must come from the payload or from timing. For brake tampering the forged payload lies far outside the normal range: bytes DATA_4 and DATA_5 move from a mean of roughly 0.5 to 58.8 and 77.5 respectively. For steering tampering the forged values sit inside the range that identifier normally carries, which is the condition we return to in Section VI-D.

B. THREAT MODEL

We assume an adversary who has already obtained write access to the CAN bus, whether through a physical diagnostic port, a compromised telematics unit or a subverted ECU, and who can transmit arbitrary frames with arbitrary identifiers. The adversary can flood the bus or forge payloads on identifiers that legitimate ECUs already use. We assume the detector observes the same bus passively and cannot authenticate senders, since the protocol provides no mechanism to do so. Attacks that leave the bus unchanged, such as purely passive eavesdropping, are outside the scope of any traffic-based detector and outside the scope of this work.

C. ARBITRATION-IDENTIFIER BIAS ANALYSIS

Before training anything we audited the dataset for shortcuts, and found a severe one. A lookup table that predicts the majority class of each arbitration ID, using no other information and no learning at all, reaches 98.9% accuracy on our test split. Its per-class F1 is 1.00 on DoS and 0.00 on both tampering classes, for a macro-F1 of 0.498.

Two things follow, and they shape the rest of this paper. Accuracy is not a meaningful headline metric on this dataset, so we report macro-F1 and per-class figures throughout. And the value of any payload-modelling component must be judged on the tampering classes, because DoS is already solved by a lookup table. We additionally train every model a second time with the arbitration ID removed from the input entirely, so that the shortcut can be eliminated rather than merely acknowledged.

The dataset also contains a high level of redundancy. 87.4% of frames are byte-identical to at least one other frame, and 80.6% of test frames have an exact duplicate in the training split. This is intrinsic to CAN, where ECUs repeat fixed status payloads at fixed intervals, but it inflates any frame-level metric and is a further reason not to lead with accuracy.

IV. PROPOSED METHOD

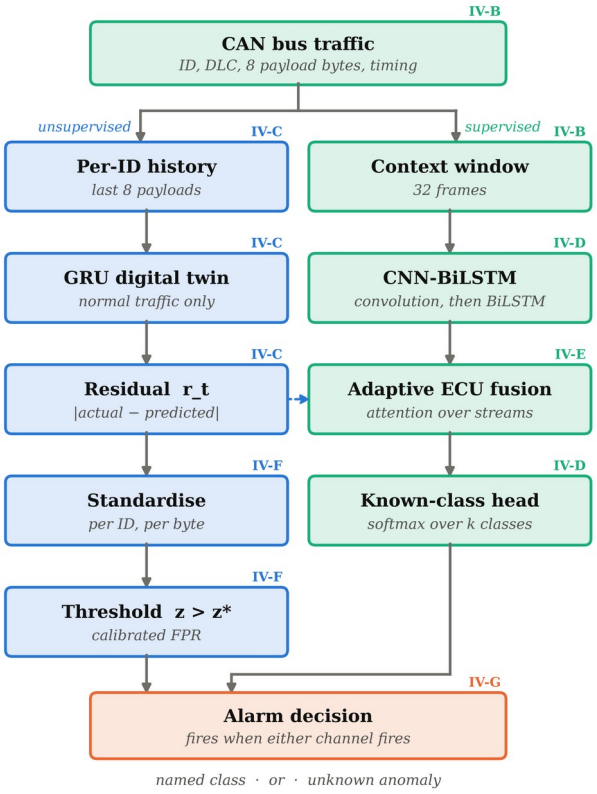


FIGURE 1. TwinShield-Fusion. Each box carries the sub-section of Section IV that specifies it. The supervised branch (right) names attacks it recognises; the unsupervised branch (left) is fitted and calibrated on normal traffic alone, and is the path that responds to an attack carrying a label the model has never seen.

A. SYSTEM OVERVIEW

Fig. 1 shows the architecture. Raw CAN traffic feeds two branches. The supervised branch builds a 32-frame context window and classifies the current frame with a CNN-BiLSTM, augmented by an attention-weighted fusion over per-ECU residual streams. The unsupervised branch passes the digital-twin residual to an anomaly gate whose threshold was fixed using normal traffic alone. A frame raises an alarm if either branch fires. The supervised branch names the attack when it recognises one; the gate can only report that something is wrong, which is the most that can honestly be said about an attack with no label.

B. CAN TRAFFIC AND CONTEXT WINDOW

Each frame is represented by its eight payload bytes scaled to [0,1], its data length code, the inter-arrival gap to the previous frame on the same identifier, and an index for the arbitration identifier itself. A single frame carries too little information to separate a forged command from a legitimate one on the same identifier, so the supervised branch reads a context window of the 32 frames preceding and including the frame under test, taken in bus order. The window is the unit of classification throughout, and its contents also determine what must be excluded under the quarantine protocol of Section V-B.

C. GRU DIGITAL TWIN

Let p_t ∈ [0,1]^8 be the scaled payload of frame t and i_t its arbitration ID. The twin is a GRU [23] with 96 hidden units and a 16-dimensional identifier embedding, which lets a single network serve every ECU. Given the eight most recent payloads observed on identifier i_t, it predicts the next one. Training minimises mean squared error against the true next payload, over normal frames from the training split only. The residual is

$$r_t = | p_t - \hat{p}_t | \qquad (1)$$

taken element-wise, so r_t is an 8-dimensional vector, one component per payload byte. On held-out normal frames the twin reaches a mean absolute error of 0.041 in the scaled units the model optimises, equivalently 10.52 on the 0–255 byte scale. That is not a small error, and it does not need to be. CAN payloads multiplex several independent signals, and some bytes are rolling counters or noisy sensor readings that are not predictable in principle. What detection requires is not a residual near zero but a residual that differs reliably between normal and tampered traffic on the same identifier.

D. CNN-BiLSTM KNOWN-CLASS HEAD

The supervised branch is a convolutional neural network (CNN) followed by a bidirectional long short-term memory (BiLSTM) network [24], a pairing chosen because the two capture complementary structure. Three one-dimensional convolutional layers read the context window and extract local byte patterns that are invariant to their position in the window, which is what a burst of injected frames looks like. The BiLSTM then reads the convolutional feature sequence in both directions, so a frame is judged against traffic that follows it as well as traffic that precedes it. A softmax head over the trained classes produces the class posterior.

This branch is the component that names an attack, and it is bounded by its label set in exactly the way described in Section I. We treat it as the recognition channel and make no claim that it generalises beyond the classes it was fitted on; the experiments in Section VI-B measure precisely that bound.

E. ADAPTIVE MULTI-ECU FUSION

Each arbitration ID is treated as a separate ECU stream. Within a context window we summarise stream s by its mean residual, giving a tensor over streams, and mask out streams absent from the window. Additive attention [25] then weights the streams:

$$e_s = v^T \tanh(W h_s + b), \quad \alpha = \mathrm{softmax}(e) \qquad (2)$$

and the fused vector Σ_s α_s h_s is concatenated with the CNN-BiLSTM embedding before the classification head. The intent is that evidence concentrates on whichever ECU is under attack rather than being diluted across the bus. We report in Section VI that on this dataset the attention contributes nothing measurable beyond the residual itself.

F. RESIDUAL ANOMALY GATE

The gate is the component that operates independently of known attacks. It reads the same residual through two views. The frame view asks how far a single frame departs from what its own ECU normally looks like, per byte:

$$z_frame(t) = max_b \left[r_\{t,b\} - \mu_\{i,b\} \right] / \sigma_\{i,b\} \quad (3)$$

where i is the frame's arbitration ID and b indexes the eight payload bytes. This is the sharp channel: a forged payload that moves one byte far out of range is visible immediately and is not averaged away by surrounding traffic. The ECU view asks how far a whole stream has drifted across the context window:

$$z_ecu(t) = max_\{s \in present\} \left[\bar{v}_s(t) - \mu_s \right] / \sigma_s \quad (4)$$

and the gate fires when the larger of the two exceeds a threshold:

$$alarm(t) = 1 \left[max(z_frame, z_ecu) > z^* \right] \quad (5)$$

The means and standard deviations are estimated on normal frames from the training split. The threshold z* is the (1−α) quantile of the score over normal frames from the validation split, with α the target false-positive rate, set to 1% in all experiments reported here. No attack frame of any class takes part in either step. This is the entire basis for describing the gate as attack-agnostic: there is no signature in it that could have been memorised, only a model of normal.

Estimating μ and σ on the twin's own training frames and reading the threshold off the same frames would understate the false-positive rate, because the twin predicts those frames better than any others. Calibrating on held-out normal frames against the same μ and σ is what makes the target rate mean something.

Standardising per identifier and per byte is essential to the design. Some ECUs, and some bytes within an ECU's payload, sit at a high baseline residual because they carry counters or noisy readings. A single global threshold fires on them continuously. Judging each byte of each ECU against its own normal behaviour is what allows one bus-wide threshold to be usable at all.

G. ALARM DECISION

The two branches are combined by disjunction. A frame raises an alarm when the supervised head assigns it to any attack class, or when the gate score exceeds z*, or both. The reported outcome differs by channel. When the supervised head fires, the system emits the class it recognised. When only the gate fires, the system emits UNKNOWN ANOMALY together with the identifier and the payload byte that produced the largest standardised residual, which is the most that can be asserted about an attack for which no label exists.

The disjunction is deliberate and its cost is explicit. Taking the union of two channels can only increase the alarm rate on normal traffic relative to either channel alone, and Section VI-C reports that increase rather than presenting the detection rate on its own.

V. EXPERIMENTAL SETUP

A. LEAKAGE-CONTROLLED SPLITTING

Consecutive CAN frames are near-duplicates, so a uniformly random split leaks almost-identical frames between training and test. We instead cut each capture into contiguous blocks of 256 frames and assign whole blocks to training, validation and test, stratified by the rarest class each block contains. A frame's context window is drawn only from inside its own block, so no frame is seen by two splits. The resulting split holds 100,352 training, 21,504 validation and 21,588 test frames; test support is 15,063 normal, 6,288 DoS, 119 brake tampering and 118 steering tampering.

A purely chronological split was considered and set aside. The tampering attacks occupy the opening 9% of the attack capture, so a chronological cut leaves the test set empty of tampering frames and makes the rarest classes unmeasurable. Blocks adjacent in time remain correlated under our scheme, which we state as a limitation of the design and revisit in Section VII.

B. LEAVE-ONE-ATTACK-OUT PROTOCOL

One attack class is designated unknown and removed from every fitting stage. Deleting its label leaves three further routes by which the class can reach a model. A frame labelled normal whose 32-frame context window contains injected brake frames has still shown the detector what the attack looks like, and a normal frame whose per-ECU history contains them has still shown the digital twin. We therefore apply four separate exclusions:

1) the detector head is k−1 way, so the class has no output index at all; 2) every training and validation frame whose context window contains a frame of the class is dropped; 3) the digital twin is refitted from scratch on normal frames whose per-ECU history is free of the class; and 4) the gate's μ, σ and threshold are taken from normal frames only, as described in Section IV-F.

The test split is left untouched and still contains all 119 held-out brake tampering frames. Because one model then has a label the others lack, accuracy and F1 are not comparable across models. We instead ask the question every deployed detector must answer, which is well defined in any label space: did you raise an alarm? We report the alarm rate on each true class, which is detection rate on the attack classes and false-positive rate on the normal class.

C. TRAINING CONFIGURATION

All detectors use a 32-frame context window, a batch size of 512, the Adam optimiser at a learning rate of 10^{-3}, and class-weighted cross-entropy to compensate for the 152:1 imbalance between normal and the tampering classes. The digital twin is trained for 5 epochs and the detectors for 6; weights are selected by best validation macro-F1 and evaluated once on the test split. All experiments run on CPU with a fixed seed of 42.

VI. RESULTS

A. CLOSED-SET CLASSIFICATION RESULTS

Table III reports macro-F1 for every model on the shared held-out test split, with and without the arbitration ID available as an input. With the ID included, the four architectures carried into the leave-one-attack-out experiment span 0.0031 macro-F1, and all five rows of Table III span 0.0041. Neither figure is evidence that one architecture is better than another; it is evidence that the benchmark has stopped measuring architecture at all. Given that a lookup table on the identifier alone already scores 98.9% accuracy, there is very little label information left for a sequence model to contribute.

TABLE III
CLOSED-SET MACRO-F1 ON THE SHARED HELD-OUT TEST SPLIT

Model	CAN ID included	CAN ID excluded
CNN	0.9968	0.9914
BiLSTM	0.9948	0.9895
CNN-BiLSTM	0.9979	0.9914
GRU twin + CNN-BiLSTM	0.9989	0.9979
TwinShield-Fusion	0.9979	0.9979

Removing the arbitration ID is more informative. There the digital-twin residual lifts the CNN-BiLSTM backbone from 0.9914 to 0.9979 macro-F1. The size of that gain should be read against the support it rests on: macro-F1 weights the two tampering classes, which hold 119 and 118 test frames, as heavily as the 15,063-frame normal class, so a single frame changing class moves a tampering F1 by roughly 0.008 and macro-F1 by roughly 0.002. A 0.0065 difference is therefore of the order of three frames. We report it as a consistent direction rather than as a precisely resolved quantity, and Section VII states what would be needed to resolve it. The ablation in Table V is not monotonic, and we read it as it stands. With the identifier included, adding the scalar twin output lowers macro-F1 from 0.9979 to 0.9958, and only the full per-byte residual recovers and exceeds the backbone at 0.9989; with the identifier excluded the ladder does rise at every step. Adaptive fusion then contributes nothing on top of the residual in either variant (0.9979 → 0.9979). The gain in this experiment comes from the residual, not from the attention layer, and we report it that way.

TABLE V
ABLATION ON THE SAME CNN-BILSTM BACKBONE (MACRO-F1)

Configuration	CAN ID incl.	CAN ID excl.
CNN-BiLSTM	0.9979	0.9914
+ digital twin (scalar)	0.9958	0.9965
+ residual vector	0.9989	0.9979
+ adaptive fusion	0.9979	0.9979

B. DETECTION OF A QUARANTINED ATTACK

Table V and Fig. 2 give the central result. With brake tampering removed from every training stage, the three supervised baselines flag 0.84%, 0.00% and 25.21% of its 119 held-out frames. TwinShield-Fusion flags all 119.

TABLE IV
LEAVE-ONE-ATTACK-OUT: BRAKE TAMPERING QUARANTINED FROM EVERY TRAINING STAGE

Model	Detected	False alarms	Known macro-F1
CNN	0.84%	0.03%	[verify]
BiLSTM	0.00%	0.08%	[verify]
CNN-BiLSTM	25.21%	0.31%	[verify]
TwinShield-Fusion	100.00%	2.26%	[verify]

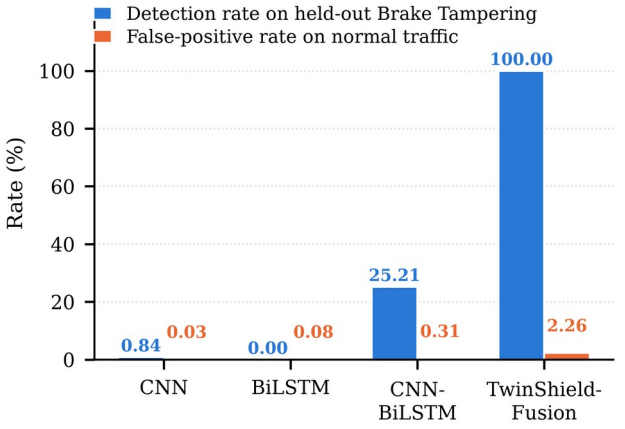


FIGURE 2. Detection of a quarantined attack class. Brake tampering was removed from the label space, from every training context window, from the digital twin's per-ECU history and from the gate's calibration set. Detection rate is measured on its 119 held-out test frames; the false-positive rate is measured on 15,063 normal test frames.

The credit belongs to a specific component, and the decomposition is worth stating. TwinShield-Fusion's supervised head contributes none of that detection—it has no label to assign—and the anomaly gate contributes all of it. The residual is what generalises here, and the classifier is what names. Attributing the result to the architecture would misplace the credit.

CNN-BiLSTM's 25.21% deserves comment, because the mechanism behind it is misattribution rather than recognition. That model also lacks an output for brake tampering. What it does is assign some of those frames to steering tampering, the other tampering class it was trained on, which happens to share an identifier family and a timing signature. The alarm is correct by accident. The two weaker baselines do not even manage that.

C. FALSE-POSITIVE COST OF THE SECOND CHANNEL

An always-on alarm detects every attack, so a detection rate reported without its false-positive rate is not a result. Ours is 2.26% on normal test traffic, against 0.03% to 0.31% for the baselines—roughly 340 false alarms per 15,063 normal frames. This is the price of the extra channel and it is not negligible. The target rate at calibration was 1%; the realised rate is higher because the supervised head's own false alarms add to the gate's, and because validation and test normal traffic are not identically distributed.

Stated per frame, 2.26% is 22.6 false alarms in every thousand normal frames. The operationally meaningful unit is alarms per unit time, which on a bus carrying f frames per second is 0.0226f. At any frame rate a production powertrain bus sustains, that is more alarms than an operator or a downstream controller could triage, and we therefore present the gate at this operating point as a research instrument rather than as a deployable alarm. The threshold is the available knob: lowering the target false-positive rate moves the system down the detection curve, and the trade-off a deployment must make is the one that curve describes. We have not measured where on it a tolerable alarm rate sits, because that depends on a per-vehicle frame rate and a per-fleet triage budget this capture does not supply.

Fig. 3 places the two experiments side by side and summarises the argument of this paper. On the closed set the four models are indistinguishable. On the quarantined class they differ by more than 74 percentage points. A closed-set score measures how well a model handles attacks somebody has already labelled, and nothing more.

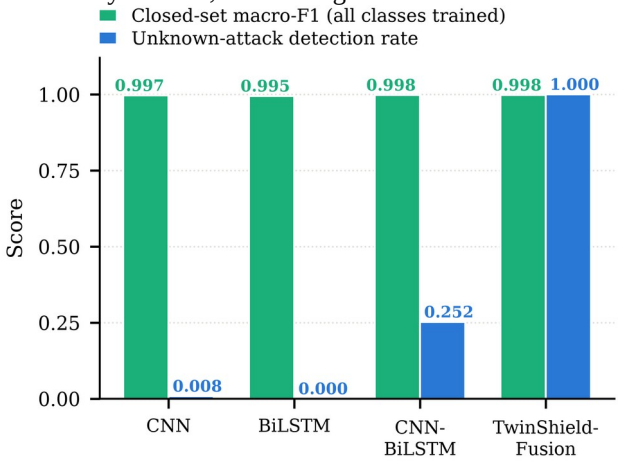


FIGURE 3. The same four models under two protocols. Closed-set macro-F1 separates them by 0.0031; detection of a quarantined attack separates them by more than 74 percentage points.

D. OPERATING CONDITION OF THE RESIDUAL SIGNAL

The gate detects an attack when that attack pushes payload bytes outside what the twin has learnt is normal for the relevant identifier. This is a real condition, not a formality, and Fig. 4 shows it being met in one case and failing in the other.

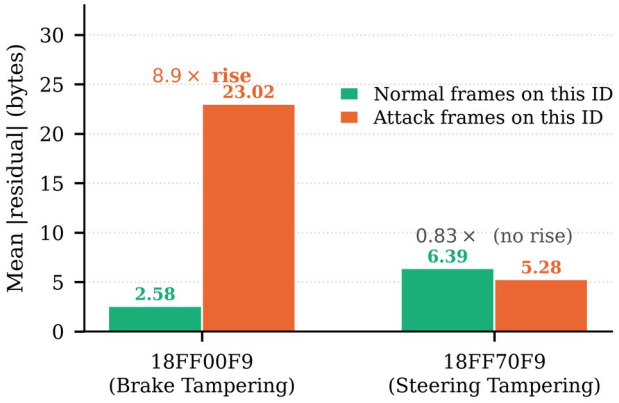


FIGURE 4. Mean digital-twin residual on the two tampering identifiers. Brake tampering raises the residual by a factor of 8.9 and is detected. Steering tampering leaves it below the normal level and is not, because the forged payloads fall inside the range the twin already accepts for that identifier.

On identifier 18FF00F9 the mean residual rises from 2.58 bytes on normal frames to 23.02 bytes under brake tampering, a factor of 8.9. The twin localises the attack precisely: bytes DATA_4 and DATA_5 carry almost all of the increase. On 18FF70F9 the mean residual under steering tampering is 5.28 bytes against 6.39 on normal traffic—lower than normal, not higher. The forged steering payloads fall inside the range the twin already accepts for that identifier.

The gate therefore cannot detect steering tampering, and this follows from the measurement rather than from a separate experiment: a score defined as a positive deviation above learnt-normal cannot fire on traffic whose residual is below that level. The boundary this places on the method is worth stating precisely. Detection does not require prior knowledge of the attack, but it does require the attack to be anomalous in the payload. Those are different conditions, and conflating them would overstate what the method does.

E. RUNTIME

A forward pass through all four models plus one gate evaluation takes 12.0 ms per frame on CPU, approximately 3 ms per model. The gate itself is two standardisations and a comparison, and its cost is negligible beside the twin's forward pass. We have not engineered the system for embedded deployment and make no claim about line-rate operation on vehicle hardware.

VII. DISCUSSION AND LIMITATIONS

The result we report is generalisation across attack classes within a single capture. It is not generalisation across vehicles, drive cycles or attack families, and those are different claims requiring a second dataset. GEM-CAN is one vehicle on one closed track, and our figures are within-capture performance.

A second boundary is worth making explicit. Brake tampering is unknown to our models, but its arbitration identifier is not. 18FF00F9 carries legitimate traffic throughout the capture, so the twin has a normal profile for it, and that profile is exactly what the residual is measured against. An attack arriving on an identifier the twin has never

observed would fall back on the unknown-ID embedding, and we have not tested that case.

A replay attack would likely evade the gate for the same reason steering tampering does. Replaying a genuine past payload produces a payload the twin finds predictable, so the residual stays low. Detecting replay requires timing or sequence modelling rather than payload modelling, and the supervised branch is better placed to provide it.

Interpretability is partial. The gate reports the identifier and payload byte carrying the largest standardised residual, which localises an alarm to an ECU and a signal, and we compute Shapley additive explanation values [26] over the supervised branch's inputs to attribute its decisions. Neither device explains why the twin finds a given payload surprising, and a full account of that would require decoding the payload into physical signals, which the vehicle's database would be needed to do.

Finally, the block-wise split leaves adjacent blocks temporally correlated, and 87.4% of frames in the dataset are byte-identical to another frame. Both inflate frame-level metrics. Neither affects the leave-one-attack-out comparison in the same way, since all four models face identical conditions there, but both should temper any reading of the absolute closed-set figures.

Four limits of the evaluation itself should be stated alongside those of the method, because each bounds how far the headline comparison can be pressed.

The experiment uses one quarantined class. Brake tampering was held out because it is the class whose residual behaviour the twin can represent; steering tampering, held out in its place, is the negative case reported in Section VI-D. A full leave-one-attack-out sweep over every class, which this dataset supports for three, would characterise the method rather than illustrate it.

Every figure comes from a single seeded run. The difference that carries the argument, 74 percentage points on the quarantined class, is far larger than any plausible seed variance; the differences in Tables III and V, of the order of a few frames, are not. Those tables should carry confidence intervals over repeated seeds before any ordering among the four architectures is asserted, and we assert none.

The baseline set is three supervised architectures. It does not include the unsupervised comparisons a reader would reasonably ask for: a plain autoencoder, a one-class model, an explicit open-set scoring rule, or a per-identifier range check, which is the simplest thing that could detect brake tampering on this capture. The one unsupervised number we do report is the twin residual acting alone, since the decomposition in Section VI-B attributes the whole of the quarantined-class detection to the gate and none of it to the supervised head.

Validation is confined to GEM-CAN. The protocol and the gate transfer to any capture carrying per-frame labels, and Car-Hacking, OTIDS, SynCAN and ROAD all qualify, but a result on one vehicle is a result on one vehicle. We state the scope rather than extrapolate past it.

VIII. CONCLUSION

Closed-set evaluation has told the CAN intrusion detection literature that the problem is close to solved. On the dataset studied here, the four architectures compared land within 0.0031 macro-F1 of one another with the arbitration identifier included, which says more about the benchmark than about the models. Under a leave-one-attack-out protocol that quarantines an attack class from the label space, the training context windows, the digital twin's history and the anomaly threshold's calibration set, the same four models separate by more than 74 percentage points. A digital-twin residual gate calibrated on normal traffic alone detects every held-out frame of the quarantined class at a 2.26% false-positive rate, while the best supervised baseline reaches 25.21%.

The mechanism has a precise boundary and we have measured it rather than asserted past it: detection requires the attack to be anomalous in the payload, and an attack whose forged values lie inside the learnt-normal range is not caught. Future work should validate across vehicles and drive cycles, and strengthen the residual representation for attacks that imitate normal payload statistics, which is where the remaining headroom lies.

REFERENCES

[1] Road Vehicles—Controller Area Network (CAN)—Part 1: Data Link Layer and Physical Signalling, ISO Standard 11898-1, 2015.
[2] K. Koscher et al., "Experimental security analysis of a modern automobile," in Proc. IEEE Symp. Security Privacy, Oakland, CA, USA, 2010, pp. 447–462.
[3] S. Checkoway et al., "Comprehensive experimental analyses of automotive attack surfaces," in Proc. 20th USENIX Security Symp., San Francisco, CA, USA, 2011.
[4] C. Miller and C. Valasek, "Remote exploitation of an unaltered passenger vehicle," Black Hat USA, Las Vegas, NV, USA, Tech. Rep., 2015.
[5] M.-J. Kang and J.-W. Kang, "Intrusion detection system using deep neural network for in-vehicle network security," PLoS ONE, vol. 11, no. 6, Jun. 2016, Art. no. e0155781.
[6] H. M. Song, J. Woo, and H. K. Kim, "In-vehicle network intrusion detection using deep convolutional neural network," Veh. Commun., vol. 21, Jan. 2020, Art. no. 100198.
[7] E. Seo, H. M. Song, and H. K. Kim, "GIDS: GAN based intrusion detection system for in-vehicle network," in Proc. 16th Annu. Conf. Privacy, Security Trust (PST), Belfast, U.K., 2018, pp. 1–6.
[8] S. F. Lokman, A. T. Othman, and M. H. Abu-Bakar, "Intrusion detection system for automotive Controller Area Network (CAN) bus system: A review," EURASIP J. Wireless Commun. Netw., vol. 2019, Dec. 2019, Art. no. 184.
[9] C. Geng, S.-J. Huang, and S. Chen, "Recent advances in open set recognition: A survey," IEEE Trans. Pattern Anal. Mach. Intell., vol. 43, no. 10, pp. 3614–3631, Oct. 2021.
[10] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," ACM Comput. Surv., vol. 41, no. 3, Jul. 2009, Art. no. 15.
[11] S. Rajapaksha, H. Kalutarage, M. O. Al-Kadri, A. Petrovski, G. Madzudzo, and M. Cheah, "AI-based intrusion detection systems for in-vehicle networks: A survey," ACM Comput. Surv., vol. 55, no. 11, Feb. 2023, Art. no. 237.
[12] B. S. Bari, K. Yelamarthi, and S. Ghafoor, "Intrusion detection in vehicle controller area network (CAN) bus using machine learning: A comparative performance study," Sensors, vol. 23, no. 7, Mar. 2023, Art. no. 3610.
[13] H. Lee, S. H. Jeong, and H. K. Kim, "OTIDS: A novel intrusion detection system for in-vehicle network by using remote frame," in Proc. 15th Annu. Conf. Privacy, Security Trust (PST), Calgary, AB, Canada, 2017, pp. 57–5709.

[14] M. Müter and N. Asaj, "Entropy-based anomaly detection for in-vehicle networks," in Proc. IEEE Intell. Vehicles Symp. (IV), Baden-Baden, Germany, 2011, pp. 1110–1115.
[15] M. H. Shahriar, Y. Xiao, P. Moriano, W. Lou, and Y. T. Hou, "CANShield: Deep-learning-based intrusion detection framework for controller area networks at the signal level," IEEE Internet Things J., vol. 10, no. 24, pp. 22111–22127, Dec. 2023.
[16] A. Taylor, S. Leblanc, and N. Japkowicz, "Anomaly detection in automobile control network data with long short-term memory networks," in Proc. IEEE Int. Conf. Data Sci. Adv. Anal. (DSAA), Montreal, QC, Canada, 2016, pp. 130–139.
[17] M. Hanselmann, T. Strauss, K. Dormann, and H. Ulmer, "CANet: An unsupervised intrusion detection system for high dimensional CAN bus data," IEEE Access, vol. 8, pp. 58194–58205, 2020.
[18] K.-T. Cho and K. G. Shin, "Fingerprinting electronic control units for vehicle intrusion detection," in Proc. 25th USENIX Security Symp., Austin, TX, USA, 2016, pp. 911–927.
[19] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," IEEE Trans. Ind. Informat., vol. 15, no. 4, pp. 2405–2415, Apr. 2019.
[20] M. E. Verma et al., "A comprehensive guide to CAN IDS data and introduction of the ROAD dataset," PLoS ONE, vol. 19, no. 1, Jan. 2024, Art. no. e0296879.
[21] M. Tavasoli, A. Sarrafzadeh, A. Karimoddini, T. Phuapaiboon, M. Khaleghi, and D. Tobias, "GEM-CAN: A real-world dataset of CAN-bus attack scenarios on an autonomous vehicle for intrusion-detection research," Data Brief, vol. 66, Jun. 2026, Art. no. 112805.
[22] M. Tavasoli et al., GEM-CAN: Real-World CAN-Bus Attack Scenarios on an Autonomous Vehicle for Intrusion Detection (Version 1) [Data set], Zenodo, 2025. [Online]. Available: https://doi.org/10.5281/zenodo.17834776
[23] K. Cho et al., "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), Doha, Qatar, 2014, pp. 1724–1734.
[24] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Comput., vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
[25] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in Proc. Int. Conf. Learn. Represent. (ICLR), San Diego, CA, USA, 2015.
[26] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Long Beach, CA, USA, 2017, pp. 4765–4774.

FIRST A. AUTHOR received the B.E. degree in computer science and engineering from [Institution], [City], [Country], in [year]. [One or two sentences on research interests, relevant project work and any awards. Replace this placeholder before submission.]

SECOND B. AUTHOR received the B.E. degree in computer science and engineering from [Institution], [City], [Country], in [year]. [One or two sentences on research interests, relevant project work and any awards. Replace this placeholder before submission.]

THIRD C. AUTHOR received the B.E. degree in computer science and engineering from [Institution], [City], [Country], in [year]. [One or two sentences on research interests, relevant project work and any awards. Replace this placeholder before submission.]